

Chapter 16

Additional Topics

Throughout this part of the book we have looked at some topics that are frequently raised whenever user stories are discussed. We've looked at how user stories differ from other requirements methods and why user stories are preferable in some cases. We've looked at some common smells, or problems, with user stories and how to correct them. In this chapter we turn our attention to several additional topics:

- handling nonfunctional requirements
- whether teams should use paper note cards or software
- the impact of user stories on the user interface
- whether user stories should be retained after they have been developed
- the relationship between bug reports and stories

Handling NonFunctional Requirements

A common stumbling block for teams getting started with user stories is their feeling that everything must be conveyed as a user story. Most projects will encounter at least a few requirements that cannot be adequately reflected as stories. Frequently these will be a system's nonfunctional requirements.

Nonfunctional requirements can address a variety of system needs. Some of the more common types of nonfunctional requirements are found in the following areas:

- performance
- accuracy
- portability

- reusability
- maintainability
- interoperability
- availability
- usability
- security
- capacity

Many nonfunctional requirements can be considered as constraints on the system's behavior. For example, it is not uncommon for a project to include a requirement such as "The system shall be written in Java." This is clearly a constraint on the design of the rest of the system. As discussed in Chapter 7, "Guidelines for Good Stories," constraints are best handled by writing the constraint on a card and annotating the card with "Constraint." In most cases, an automated test can be written (and run at least daily) to ensure compliance with a constraint. A few constraints cannot be realistically tested or are not worth testing. The constraint "The system shall be written in Java" comes to mind. There are certainly easy ways to ensure this constraint is satisfied.

Examples of some common constraints are shown in Table 16.1. Beyond writing constraints on a card, if there are additional nonfunctional requirements that a system must adhere to, then communicate them in whatever form is appropriate or traditional. If, for example, the project can benefit from a data dictionary showing the size and type of all variables in the system, then create a data dictionary.

Table 16.1 *Sample constraints written for a variety of common nonfunctional requirements.*

Area	Sample Constraint
Performance	80% of database searches will return results to the screen in less than two seconds.
Accuracy	The software will correctly predict the winner of a football game at least 55% of the time.
Portability	The system shall not make use of any technology that would make it difficult to port to Linux.
Reusability	The database and database access code will be reusable in future applications.

Table 16.1 *Sample constraints written for a variety of common nonfunctional requirements. (Continued)*

Area	Sample Constraint
Maintainability	Automated unit tests must exist for all components. Automated unit tests must be run in their entirety at least once every 24 hours.
Interoperability	The system shall be written in Java. All configuration data shall be stored in XML files. Data shall be stored in MySQL.
Capacity	The database will be capable of storing 20 million members on the specified hardware while still meeting performance objectives.

Paper or Software?

Even more common than being asked “paper or plastic?” at the grocery store is the question of whether stories should be written on paper note cards or stored in a software system. Many in the Extreme Programming community advocate the use of paper note cards because of their simplicity. Extreme Programming places a premium on simple solutions, and paper note cards are definitely simple. Additionally, cards encourage interaction and discussion. They can be placed on a table in various formations during planning, they can be stacked and sorted, they can be carried into any meeting, and so on.

On the other hand, there are software products designed specifically for tracking stories (VersionOne,¹ XPlanner,² Select Scope Manager³) as well as general purpose software that can be used with stories (spreadsheets, defect trackers and wikis).

One of the main advantages that cards have over software is that their low-tech nature is a constant reminder that stories are imprecise. When shown in software, stories may take on the appearance of IEEE 830-style requirements and those writing stories may add additional, unnecessary detail because of that.

The typical note card can hold a limited amount of writing. This gives it a natural upper limit on the amount of text. This limitation does not exist in most software alternatives. On the other hand, a common practice among those

1. See www.versionone.net.

2. See www.xplanner.org

3. See www.selectbs.com/products/products/select_scope_manager.htm.

using note cards is to write some sample acceptance tests for a story on the back of the card. In many cases, the size of the card can work against it when writing the test cases.

Choosing Software at ClickTactics

ClickTactics is a marketing solutions provider that writes web-accessible software components. They started with note cards but switched to software, V1:XP from VersionOne.

Mark Mosholder, Senior Product Manager at ClickTactics, says one reason for the change is that their sales force and upper management are distributed across multiple sites. With remote stakeholders they could not say “go look at the board” so they spent a lot of time updating senior management and other remote stakeholders. Also, when using note cards they had problems with cards occasionally getting lost and then found weeks later in a pile under a desk.

Keeping stories in the VersionOne software allowed ClickTactics to promote their use of XP as a sales tool. Using the software, they give some customers limited visibility into their iterations. They then promote the speed with which they can turn around new versions for customers by telling them, “We can get you new functionality in three weeks.”

Mark says there have been no drawbacks to their decision and says he would make the same decision again.

A project pursuing ISO (International Organization for Standardization), or similar, certification that requires traceability from a requirement statement down through code and tests will probably favor software. It should be possible to achieve ISO certification with handwritten note cards, but putting in place and demonstrating adequate change control procedures over a deck of cards probably outweighs the other advantages cards may have.

Similarly, a team that is not collocated will probably prefer software over note cards. When one or more developers, or especially the customer, is remote, it is too hard to work with paper.

An additional advantage to note cards is that they are very easy to sort and can be sorted in a variety of ways. A collection of stories can be sorted into high, medium and low priority piles. Or it could be sorted into more precise order with the first story higher priority than the second and the second higher priority than the third and so on.

Unlike many of the most devout supporters of either technique, my opinion is that both approaches can be appropriate. I recommend starting with cards

and seeing how they work in your environment. Then, if there's a compelling reason to use software, switch.

Using a Wiki at Diaspar Software Services

J. B. Rainsberger is the founder of Diaspar Software Services, a software development and consulting company. As a consultant, J. B. cannot always be on-site with his customers. In those situations, J. B. uses a wiki to improve communication between himself and his remote customer. A wiki is a set of special web pages that can be edited by anyone viewing the page. J. B. and Diaspar Software use FitNesse as their wiki. Rather than writing a story card for each story they create a new page for each story.

J. B. reports that this worked extremely well on a recent small project. As he had questions about a story he would note the question on the page and add the text "todo." Several times a week his customer would check the wiki, search for "todo," and answer the questions. Urgent issues were handled by phone; but because of the efficiencies of using a wiki there were surprisingly few urgent issues. J. B. has worked with paper note cards on other projects but reports that on this project, with its remote customer, there was never a time when he wished he had the stories on cards as well as in the wiki.

Although J. B. uses the FitNesse wiki for writing executable tests for every project, he points out that "having everyone in the room makes it unnecessary to put stories on a wiki."

User Stories and the User Interface

It has been pointed out that agile methods largely ignore issues of designing the user interface. To some extent this is understandable: Agile processes are highly iterative, while traditional approaches to user interface design have been big-bang with heavy reliance on upfront design. It is important to understand the potential risks in pursuing an agile story-based approach for an application with a significant or important user interface.

It is one of the tenets of agile development that we may iteratively refine a system. User stories allow us to defer conversations until shortly before the developers are ready to add support for the story to the program. Sometimes deferring these conversations causes the developers to slightly rework existing parts of an application; but the belief is that the cost of these slight reworkings is more than justified by the savings in not holding requirements discussions

about features that will later be dropped, and by the benefits of allowing the customer to steer the product with many smaller course corrections.

If these changes happen beneath the user interface of an application then this belief may be true. However, what happens when these changes affect the user interface? Larry Constantine (2002) writes that:

For user interfaces, the architecture—the overall organization, the navigation, and the look-and-feel—must be designed to fit the full panoply of tasks to be covered. When it comes to the user interface, later refinement of the architecture is not as acceptable because it means changing the system for users who have already learned or mastered an earlier interface. Even small adjustments in the placement or form of features can be problematic for users.

This means that if the user interface will be important to the success of our product, then we might need to think about the user interface right from the start. If changing the user interface will cause problems for users then there is probably an unwritten constraint on the project—“Change the user interface as little as possible once it’s started.”

Constantine and Lockwood (2002) propose agile usage-centered design as a solution. Agile usage-centered design is driven by essential use cases or task cases rather than user stories. However, we can replace essential use cases with stories, which leads to the following story-based variation on Agile Usage-Centered Design:

1. Perform user role modeling.
2. Trawl for high-level user stories.
3. Prioritize stories.
4. Refine high- and medium-priority stories.
5. Organize stories into groups.
6. Create a paper prototype.
7. Refine the prototype.
8. Start programming.

The first step is to hold a user role modeling session, exactly as described in Chapter 3, “User Role Modeling.” To complete the next few steps, initiate a story-writing workshop as described in Chapter 4, “Gathering Stories.” In the workshop focus initially on capturing the highest-level stories, probably no more than two dozen.

Next, prioritize the high-level stories into three groups: High-priority stories must be in the upcoming release, medium-priority stories are desired for the upcoming release, and low priority stories can be deferred until a subsequent release. Put the low priority stories aside while refining the high- and medium-priority stories into smaller stories. These stories should be the size you will work with in planning the release.

Next, organize the high- and medium-priority stories into groups that have a high likelihood of being performed together. Then, draw prototypes on paper for each group of stories. After creating the paper prototypes, show them to users (or user proxies if necessary) and refine the prototypes based on their comments.

If you add these steps to your project, remember to keep the process as light-weight as you can. Some of the stories that were identified and that are being prototyped into a user interface could end up being removed from the project before they get developed. Avoid spending any more time than necessary. For most applications, this may entail somewhere from a few days to no more than a few weeks (for commercial software with remote users).

Write Two

A few years ago I was working on a project and we brought Ward Cunningham in to consult and assess the project. At the time the team was struggling with questions about the user interface. There were many hot-and-heavy debates about whether users would prefer a browser-based interface or whether they'd prefer a native application. Our marketing group had asked the users but we weren't confident that they had surveyed enough users or that they'd done the research the right way.

Ward settled the question by telling us "Write two user interfaces." His logic was that neither would be hard to write and between the two of them, they would keep the application's middle-tier honest. With two user interfaces no functionality would be inappropriately moved to the client tier if doing so meant that functionality would have to be written twice.

Ward's suggestion was right. We, of course, failed to listen, thinking it would be too expensive to develop two complete user interfaces. When the product was finished our customers let us know that we had indeed chosen the wrong user interface technology. And a project was quickly begun to add a second user interface.

Retaining the Stories

Arguments over whether stories should be retained are common. On one side are those who say that the pleasure of ripping up a completed card outweighs any value of retaining the card. On the opposite side is the more conservative crowd who would rather save the story than risk throwing it away and then needing it later.

If you're using software to hold your stories there is very little reason to get rid of completed stories. You may get some pleasure and feelings of closure from deleting an electronic story but it's probably not as great as the visceral pleasure of ripping a physical card in half.

If you're using paper note cards, then there can indeed be a tangible pleasure associated with finishing a card and ripping it in half. I've used cards to guide the writing of this book and each time I complete a new section or revision I rip up the card. However, when I'm working on a software project, rather than a book, I prefer to retain the cards, archiving them on a shelf with a rubber band.

Over the years I've been involved in a handful of situations where I've been thankful that I retained the requirements. Here are some of the cases:

- The company I was working for was being bought. The acquiring company was intrigued by our lightweight software development process but they had their own heavily waterfall-oriented approach with numerous gates and sign-off points each project had to pass through. Because I was able to actually show them our process (from stories to code and tests) they allowed us to continue with our process and not adopt the company-wide standard. Even better, we were eventually able to make some inroads in spreading our process into other divisions of the company.
- On a number of occasions I've been involved in completely rewriting commercial products. In one case the first version of a product was a commercial failure because of some poor technology choices. The product was completely rewritten and became a moderate success. Another product was a phenomenal success as a client-server application, and five years later the company wanted it rewritten and available on the web. Whenever there were even outdated stories or requirements, they were useful to have around.
- In yet another situation I was involved with a small startup that was trying to close a deal with a much larger company. If the deal was closed it would push the company into profitable territory, and the boss was also promising high five-digit bonuses to all of the developers. We were asked to "pro-

vide a copy of the requirements.” I started to go down the road of describing how we didn’t really focus on written requirements and that we instead focused on conversation and collaboration. I could sense the conversation was not going to end well and that company profitability and the large developer bonuses were evaporating. I shifted gears and told them how we wrote requirements in the form of user stories. They liked the idea. Fortunately our stories were stored electronically on that project. We cut them from the system they were in, pasted them into a Word document, added a cover page and a signature page and everyone was happy.

Considering the variety of occasions on which I’ve found it useful to have retained stories, my recommendation is that you do so as well. If you’re using software, either keep the software installed or print a report from it and file the report somewhere. If you’re using cards, either save the cards themselves or photocopy them three at a time onto paper.

Stories for Bugs

A very common question is the relationship between stories and bug reports. What I’ve found to work best is to consider each bug report its own story. If fixing a bug is likely to take as long as a typical story, that bug can be treated just like any other story. However, for bugs that the team expects to be able to fix quickly, you should combine the bugs into one or more stories. With cards you can easily do this by stapling the story cards together along with a cover story card. Then, for planning purposes, the collection of bugs may be treated as a single story.

What About Colors?

Some teams favor using cards of different colors to indicate the type of story on the card. For example, traditional white cards could be used for normal stories. Red cards could be used for bugs, blue cards could be used for engineering tasks and so on.

To me, using colored cards always seems like a good idea in theory; but in practice is not worth the extra hassle. Instead of just keeping a supply of white cards in my pocket, I have to make sure to always carry a few of each color. If I run out and write the story on whatever is available I then need to rewrite the story later. But if you think colors will help you organize your stories, try it. I’ll stick to what’s simple—a huge pile of white cards.

Summary

- Nonfunctional requirements (such as for performance, accuracy, portability, reusability, maintainability, interoperability, capacity and so on) can often be handled by creating constraint cards. If the system has requirements that are more complex than this, supplement your user story approach with whatever additional format or technique best expresses those requirements.
- Neither note cards nor a software system is the best way to write stories in all cases. Match the tool to the project and team.
- An iterative process can lead to iterative changes in the user interface. Users, who become accustomed to very specific aspects of the interface, do not like user interface changes that affect how they have learned to operate the software. Consider adding some practices from Agile Usage-Centered Design to avoid iterating the user interface.
- There's a certain joy in ripping up a story card as soon as it's complete. But there are also reasons to retain the cards. Err on the safe side and retain the stories.
- Staple small bug reports together with a cover story card and treat them as a single story.

Developer Responsibilities

- You are responsible for suggesting and using alternative techniques and methods for expressing requirements when appropriate.
- You share a responsibility to decide what is right for your project: note cards or a software system.
- You share a responsibility for understanding the advantages and disadvantages of considering the overall user interface at the start of the project.

Customer Responsibilities

- You are responsible for requesting or using alternative techniques and methods for expressing requirements if you do not feel user stories are accurately reflecting some part of the requirements.
- You share a responsibility to decide what is right for your project: note cards or a software system.
- You share a responsibility for understanding the advantages and disadvantages of considering the overall user interface at the start of the project.

Questions

- 16.1 How should you handle a requirement for a system to scale up to use by 1,000 concurrent users?
- 16.2 Do you prefer to write stories on note cards or in a software system? Defend your answer.
- 16.3 What impact does an iterative process have on the user interface of an application?
- 16.4 Give some examples of systems that could benefit from more upfront consideration of the user interface than is typically given on an agile project.
- 16.5 Do you recommend retaining or disposing of stories once the story has been developed? Defend your answer.

This page intentionally left blank

PART IV

An Example

Part IV brings everything together in a comprehensive example. In the chapters that follow, you'll meet South Coast Nautical Supplies and some of its employees as they create a website to augment their print catalog. You'll have the opportunity to identify user roles, write stories with Lori (South Coast's vice president of Sales and Marketing and the customer for this project), estimate the stories, create a release plan and then write some acceptance tests for the stories in the initial plan.

This page intentionally left blank